

1 Functions and Local Scopes

We extend the language with a new category — *declarations* ($\mathcal{D}$), which consists of local variable declarations and function declarations. At the expression level we add *scope expressions* ($\mathcal{S}$), nested scopes and function calls. Finally, the category of programs $\mathcal{P}$ is scope expression:

$\mathcal{S}$	$=$	$\mathcal{D}^* \mathcal{E}$	— scope expression
$\mathcal{E}$	$+=$	$(\mathcal{S})$	— nested scope
		$\mathcal{X} (\mathcal{E}^*)$	— function call
$\mathcal{D}$	$=$	var $\mathcal{X}$	— local variable definition
		fun $\mathcal{X} (\mathcal{X}^*) \{ \mathcal{S} \}$	— function definition
$\mathcal{P}$	$=$	$\mathcal{S}$	— program

1.1 Concrete Syntax

On the concrete syntax level we stipulate the following conventions:

1. the expression in scope expression is optional; if no expression is explicitly specified then "**skip**" is assumed;
2. local variable definition has to be terminated by a semicolon, for example

```
var x;
```

3. multiple variable names can be specified in a single definition, for example

```
var x, y, z;
```

is equivalent to

```
var x;  
var y;  
var z;
```

4. an optional initializer can be specified for a local variable definition; the initializers are reified into sequential assignments, preserving their order, for example

```
var x = 3;  
var y = x + 5, z = x + y;
```

is equivalent to

```
var x;  
var y;  
var z;  
  
x := 3;  
y := x + 5;  
z := x + y
```

5. scope expressions are implicitly assumed in the bodies of loops and branches of conditional expressions;
6. in "do..while" expression the scope of body's *immediate* definitions is implicitly extended to enclose the whole expression, thus

do var i ; **read** (i) **while** $x > 0$ **of**

is equivalent to

(**var** i ;
do read (i) **while** $x > 0$ **od**)

7. an implicit scope expression is assumed in the initialization part of "for"-loop; the scope of its immediate definitions is extended to the whole expression as well, thus

for var i ; $i := 0, i < 10, i := i+1$ **do write** (i) **od**

is equivalent to

(**var** i ; **for** $i := 0, i < 10, i := i+1$ **do write** (i) **od**)

1.2 Well-formedness

We assume functions to always return a values; thus, we need a way to materialize a void into some default value " $\perp$ ". We do this by introducing a new type of attribute — "**Weak**" — and adding the following set of rules to the inference system we used to ensure/restore expression well-formedness (see Fig. 1).

Weak : $x, \quad x \in \mathcal{X}$	Weak : $z, \quad z \in \mathbb{N}$
$\frac{\text{Val} : l, \quad \text{Val} : r}{\text{Weak} : l \oplus r}$	Weak : skip ; $\perp$
$\frac{\text{Ref} : l, \quad \text{Val} : r}{\text{Weak} : l := r}$	Weak : read (x); $\perp$
$\frac{\text{Val} : e}{\text{Weak} : \text{write}(e); \perp}$	$\frac{\text{Val} : e, \quad \text{Void} : s}{\text{Weak} : \text{while } e \text{ do } s \text{ od}; \perp}$
$\frac{\text{Val} : e, \quad \text{Void} : s}{\text{Weak} : (\text{do } s \text{ while } e); \perp}$	

Figure 1: Well-formedness: additional rules for the old kinds of expressions

We also need to specify the inference rules for the *new* kinds of expressions (see Fig. 2), and, finally, the rules for definitions (see Fig. 3).

The top-level well-formedness condition for the while program p (which is now a scope expression) is

$$\begin{array}{c}
\frac{\mathbf{Val} : e_1, \dots, \mathbf{Val} : e_k}{\mathbf{Val} : f(e_1, \dots, e_k)} \quad \frac{\mathbf{Val} : e_1, \dots, \mathbf{Val} : e_k}{\mathbf{Weak} : f(e_1, \dots, e_k)} \quad \frac{\mathbf{Val} : e_1, \dots, \mathbf{Val} : e_k}{\mathbf{Void} : \mathbf{ignore}(f(e_1, \dots, e_k))} \\
\frac{\vdash^{\mathcal{D}^*} d, \quad a : e}{a : d \ e}, \quad d \in \mathcal{D}^* \\
\frac{a : e}{a : \{ e \}}
\end{array}$$

Figure 2: Well-formedness: additional rules for the new kinds of expressions

$$\begin{array}{c}
\vdash^{\mathcal{D}^*} \epsilon \quad \frac{\vdash^{\mathcal{D}} d, \quad \vdash^{\mathcal{D}^*} ds}{\vdash^{\mathcal{D}^*} d \ ds} \\
\vdash^{\mathcal{D}} \mathbf{var} \ x \quad \frac{\mathbf{Weak} : e}{\vdash^{\mathcal{D}} \mathbf{fun} \ f \ (\dots) \ \{ e \}}
\end{array}$$

Figure 3: Well-formedness: additional rules for definitions

$$\mathbf{Void} : p$$

1.3 Semantics

We now present the big-step operational semantics for functions and scopes. First, we introduce a shortcut for a multiple substitutions into a state: for a lists of variable names $x \in \mathcal{X}^*$ and values $v \in \mathcal{V}^*$ of equal lengths we define

$$\sigma[x_i \leftarrow v_i] = \sigma[x_1 \leftarrow v_1] \dots [x_k \leftarrow v_k]$$

Then, we add a new kind of value — a functional value:

$$\mathcal{V} + = \mathcal{X}^* \mapsto \mathcal{C}$$

The simplest form of semantics for function calls could be as follows: assume we know that f is a function with arguments $a_1, \dots, a_k$ and body b ; then we evaluate its call $f(e_1, \dots, e_k)$ as follows:

$$\frac{c \xRightarrow{e_1 \dots e_k}^* \langle \langle \sigma', \omega' \rangle, v \rangle \quad \langle \sigma'[a_i \leftarrow v_i], \omega' \rangle \xRightarrow{b} c''}{c \xRightarrow{f(e_1, \dots, e_k)} c''}$$

Thus, however, would describe *dynamic binding* for functions, while our goal to have a semantics with *static binding*.

So, we modify the definition of state as follows:

$$\Sigma = (2^{\mathcal{X}} \times (\mathcal{X} \rightarrow \mathcal{V}))^+$$

Now a state is a non-empty list of *scopes*; in each scope we have a set of scope variables and a local state. The rightmost element of a state corresponds to the *global* state; all other elements correspond to properly ordered list of enclosing scopes for a given point in a program.

We need to redefine two primitives for states: those for reading and assigning values variables. For reading:

$$((L, s)\sigma)(x) = \begin{cases} sx & , \quad x \in L \\ \sigma(x) & , \quad x \notin L \end{cases}$$

For assigning:

$$((L, s)\sigma)[x \leftarrow v] = \begin{cases} (L, s[x \leftarrow v])\sigma & , \quad x \in L \\ (L, s)(\sigma[x \leftarrow v]) & , \quad x \notin L \end{cases}$$

With these primitives redefined all existing semantic rules can be preserved; however, we need to put additional requirements for some rules (assignment, reading from a variable, reading from a world) to ensure that we do not deal with functional values.

Now we need some new rules describing the semantics of new constructs (definitions and function calls). For scope expressions, the following “structural” rules are sufficient:

$$\frac{\text{enterScope } c \xRightarrow[\mathcal{D}^*]{d} c' \quad c' \xRightarrow{e} c''}{c \xRightarrow[\mathcal{D}^*]{de} \text{leaveScope } c''} \quad [\text{SCOPE}]$$

$$\frac{c \xRightarrow{e} c'}{c \xRightarrow[\mathcal{D}^*]{(e)} c'} \quad [\text{LOCALSCOPE}]$$

We describe the primitives **enterScope** / **leaveScope** below; the transition “ $\xRightarrow[\mathcal{D}^*]{}$ ” is, again, described with obvious structural rules:

$$c \xRightarrow[\mathcal{D}^*]{\epsilon} c \quad [\text{DEFSEMPTY}]$$

$$\frac{c \xRightarrow[\mathcal{D}]{d} c' \quad c' \xRightarrow[\mathcal{D}^*]{ds} c''}{c \xRightarrow[\mathcal{D}^*]{dds} c''} \quad [\text{DEFSNONEMPTY}]$$

The interesting part is the relation “ $\xRightarrow[\mathcal{D}]{}$ ”:

$$c \xRightarrow[\mathcal{D}]{\text{var } x} \text{addName } x \ 0 \ c \quad [\text{DEFVAR}]$$

$$c \xRightarrow[\mathcal{D}]{\mathbf{fun} \ f \ (x_1 \dots x_k) \ \{ e \}} \mathbf{addName} \ f \ (x_1 \dots x_k \mapsto e) \ c \quad [\text{DEFUN}]$$

where the primitive ”**addName**” is defined as follows:

$$\mathbf{addName} \ x \ v \ \langle (L, s) \sigma, \omega \rangle = \langle (L \cup \{x\}, s[x \leftarrow v]) \sigma, \omega \rangle$$

and we introduce the following shortcut for adding multiple names at once:

$$\mathbf{addName}^* \ \langle x_1, x_1 \rangle \dots \langle x_k, v_k \rangle \ c = \mathbf{addName} \ x_k \ v_k \ (\dots (\mathbf{addName} \ x_1 \ v_1 \ c) \dots)$$

Now we define the primitives **enterScope** / **leaveScope** :

$$\begin{aligned} \mathbf{enterScope} \ \langle \sigma, \omega \rangle &= \langle (\emptyset, \Lambda) \sigma, \omega \rangle \\ \mathbf{leaveScope} \ \langle (L, s) \sigma, \omega \rangle &= \langle \sigma, \omega \rangle \end{aligned}$$

Finally, we need to define the semantics for function calls:

$$\begin{aligned} &\sigma f = (\{a_i\}_{i=1}^k \mapsto e) \\ &\langle \sigma, \omega \rangle \xRightarrow[e_1 \dots e_k]{} \left\langle c', \{v_j\}_{j=1}^k \right\rangle \\ &\quad \forall j \in \{1..k\}. v_j \in \mathbb{Z} \\ &\frac{\mathbf{addName}^* \ \langle a_1, v_1 \rangle \dots \langle a_k, v_k \rangle \ (\mathbf{enterFunction} \ c') \xRightarrow{e} \langle c'', w \rangle}{\langle \sigma, \omega \rangle \xRightarrow[f(e_1 \dots e_k)]{} \langle \mathbf{leaveFunction} \ c' \ (\mathbf{global} \ c''), w \rangle} \quad [\text{CALL}] \end{aligned}$$

The primitives ”**enterFunction** / **leaveFunction** / **global**” are defined as follows:

$$\begin{aligned} \mathbf{enterFunction} \ \langle \sigma, \omega \rangle &= \langle \mathbf{enterScope} \ (\mathbf{global} \ \sigma), \omega \rangle \\ \mathbf{leaveFunction} \ \langle (L', s') \varepsilon, \omega \rangle \ (L, s) &= \langle (L, s) \varepsilon, \omega \rangle \\ \mathbf{leaveFunction} \ \langle (L', s') \sigma, \omega \rangle \ (L, s) &= \langle (L', s') (\mathbf{leaveFunction} \ \sigma \ (L, s)), \omega \rangle, \ \sigma \neq \varepsilon \\ \mathbf{global} \ (L, s) \varepsilon &= (L, s) \\ \mathbf{global} \ (L, s) \sigma &= \mathbf{global} \ \sigma, \ \sigma \neq \varepsilon \end{aligned}$$